Indian Statistical Institute
Kolkata

Scientific Machine Learning For Computational Physics: Neural Operator Benchmarking on Darcy Flow Resolution Generalization

Summer School Project Report

Submitted by

Arjyadeb Sengupta
Arka Dhar
Rajdeep Pal

Under the Mentorship of

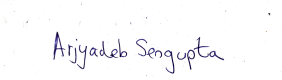
Priyobrata Mondal
Senior Research Scholar
Indian Statistical Institute, Kolkata

Summer School Programme
Indian Statistical Institute
Kolkata
July 2026

Declaration

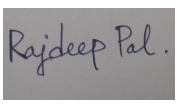
We hereby declare that the work entitled **"Scientific Machine Learning For Computational Physics: Neural Operator Benchmarking on Darcy Flow Resolution Generalization"** was carried out by us during the Summer School Programme at the **Indian Statistical Institute, Kolkata**, under the mentorship of **Priyobrata Mondal (Senior Research Scholar)**.

This report represents our original work except where due acknowledgement has been made through appropriate references. To the best of our knowledge, this work has not been submitted elsewhere for the award of any degree, diploma or other academic qualification.



Arjyadeb Sengupta

Arka Dhar



Rajdeep Pal

Acknowledgement

We sincerely thank the **Indian Statistical Institute, Kolkata**, for providing us with the opportunity to participate in the Summer School Programme.

We express our heartfelt gratitude to our mentor, **Priyobrata Mondal (Senior Research Scholar)**, for his invaluable guidance, continuous encouragement, insightful discussions and constructive suggestions throughout the course of this project.

We also thank the faculty members, research scholars and staff of the Indian Statistical Institute for providing an excellent academic and research environment. Finally, we acknowledge our families, friends and fellow participants for their constant encouragement and support.

Arjyadeb Sengupta
Arka Dhar
Rajdeep Pal

Contents

0.1 Abstract

Neural operators have emerged as a powerful class of deep learning models for approximating solution operators of partial differential equations (PDEs), offering discretization-invariant learning, rapid inference, and strong generalization across varying spatial resolutions. Unlike conventional numerical solvers, which require repeated computationally intensive discretizations for different initial or boundary conditions, neural operators directly learn mappings between infinite-dimensional function spaces, enabling efficient surrogate modeling of complex physical systems. Over recent years, the field has evolved from classical architectures such as DeepONet and the Fourier Neural Operator (FNO) to graph-based methods and Transformer-inspired attention mechanisms designed to improve geometric flexibility, long-range dependency modeling, and computational scalability.

This study presents a comprehensive comparative evaluation of ten representative neural operator architectures spanning three major categories: classical and spectral methods (DeepONet), graph-based operators (MAGNO), and attention-driven Transformer-based frameworks (GNOT, MPNOT, FActFormer, Transolver, LinearNO, and CATO). In addition, a preliminary prototype of a novel neural operator architecture developed in this work is introduced for exploratory assessment. All models are benchmarked on the two-dimensional Darcy Flow problem at a spatial resolution of 16×16 under a unified experimental framework. Performance is evaluated using training loss, testing loss, training time, and inference time to quantify the trade-offs between predictive accuracy and computational efficiency.

By systematically comparing operator-learning paradigms with differing representational backbones and computational complexities, this work highlights the strengths and limitations of existing neural operator families for low-resolution PDE modeling. The resulting analysis provides practical guidelines for selecting appropriate neural operator architectures according to accuracy, computational cost, and scalability, while establishing a unified baseline for future research on efficient operator learning in scientific machine learning.

0.2 Introduction

Modeling continuous physical systems governed by partial differential equations (PDEs) traditionally relies on numerical discretization schemes such as finite element or finite difference methods. While highly accurate, these conventional solvers demand significant computational resources when evaluated repeatedly across varying initial conditions, boundary conditions, or system parameters. Standard deep learning models mitigate this issue by acting as surrogates, yet they remain fundamentally tied to fixed computational grids and struggle to generalize across differing spatial resolutions. Neural operators resolve these limitations by mapping between infinite-dimensional function spaces, learning the underlying solution operator rather than a single discretized instance. This functional parameterization enables zero-shot super-resolution, discretization invariance, and accelerated inference on complex continuous domains.

Neural operator architectures can be systematically grouped into three primary topological families based on their underlying representation backbones: classical/spectral architectures, graph-based operators, and attention-driven Transformer frameworks.

Classical and spectral approaches establish operator learning through explicit functional decompositions or decoupled network structures. The Deep Operator Network (DeepONet) leverages the universal approximation theorem via separate Branch and Trunk networks to process input functions and domain coordinates independently at sensor points, scaling with $\mathcal{O}(N \cdot p)$ complexity. Alternatively, the Fourier Neural Operator (FNO) conducts global convolutions in frequency space using learned spectral kernels and Fast Fourier Transforms, achieving an efficient $\mathcal{O}(N \log N)$ operational complexity on regular spatial grids, making it optimal for fluid dynamics and structured flow benchmarks.

Attention-driven and Transformer-based neural operators integrate self-attention mechanisms to effectively capture long-range functional dependencies across irregular or high-dimensional domains. The General Neural Operator Transformer (GNOT) and Message Passing Neural Operator Transformer (MPNOT) combine graph message passing with attention, incurring quadratic costs of $\mathcal{O}(N^2)$ and $\mathcal{O}(E+N^2)$ respectively to model fine-scale and global dynamics on non-uniform meshes. To scale attention to higher-resolution domains, sub-quadratic variants optimize the kernel computation: Transolver uses adaptive physics-aware slicing to achieve $\approx \mathcal{O}(N \log N)$ complexity, Factorized Attention Neural Operators (FActFormer) factorize self-attention to scale at $\approx \mathcal{O}(N\sqrt{N})$, Linear Neural Operators (LinearNO) utilize linear attention mechanisms yielding $\mathcal{O}(N)$ complexity for high-resolution evaluation, and Cross-Attention Neural Operators (CATO) enable multi-resolution transfers across mixed discretizations at $\mathcal{O}(N \cdot M)$ complexity. We have additionally developed a trial version of a new architecture. This project pro-

vides a comprehensive comparative evaluation across these ten representative neural operator models to delineate their architectural trade-offs. By systematically analyzing the training-loss curve, testing-loss curve, the inference time and the training time, this study establishes clear guidelines for selecting optimal neural operator frameworks for 8 by 8 resolution 2D Darcy Flow.

0.3 Literature Review

The development of neural operators originated with the Deep Operator Network (DeepONet), introduced by Lu et al. in 2019 and published in Nature Machine Intelligence in 2021, establishing continuous operator learning by utilizing decoupled Branch and Trunk multilayer perceptrons (MLPs) to map input functions and coordinate query points based on the universal approximation theorem at $\mathcal{O}(N \cdot p)$ complexity on sensor point discretizations. However, DeepONet's primary limitation was a weak spatial inductive bias, leading to slow execution and poor performance on large or high-dimensional spatial domains. To overcome this spatial inefficiency, the Fourier Neural Operator (FNO) was introduced by Li et al. in 2020 (ICLR 2021), parameterizing integral kernels in Fourier space and executing global convolutions via Fast Fourier Transforms (FFT) at $\mathcal{O}(N \log N)$ complexity to capture global spatial interactions with state-of-the-art accuracy on structured benchmarks like Navier-Stokes, Darcy flow, and Burgers equations, though its strict requirement for regular spatial grids left it unable to natively model complex or irregular boundary geometries. To resolve FNO's geometric limitations and support arbitrary boundary shapes, the Vanilla Graph Neural Operator (Vanilla GNO) was introduced by Li et al. in 2020 (NeurIPS 2020), mapping spatial domain points onto unstructured graph topologies to approximate integral operators using localized neighborhood message passing at $\mathcal{O}(E)$ complexity, though its constrained receptive field made it difficult to capture long-range physical dependencies and caused over-smoothing in deep networks. To address Vanilla GNO's limited receptive field and multi-scale physical dynamics, the Multi-Scale Adaptive Graph Neural Operator (MAGNO) was developed in 2022 using a multiscale hierarchical graph structure that processed features at $\mathcal{O}(E \log N)$ complexity to capture fine-grained local details and coarse-scale global trends across adaptive meshes, albeit at the cost of significant algorithmic overhead required to construct and manage multi-scale graph hierarchies. To model global long-range physical interactions on unstructured domains without constructing complex graph hierarchies, attention mechanisms were incorporated into operator architectures, beginning with the General Neural Operator Transformer (GNOT) proposed in 2023 by Hao et al. (ICML 2023), combining graph spatial operators with Transformer self-attention to achieve geometry-resolution invariance on unstructured domains at $\mathcal{O}(N^2)$ complexity. Concurrently, the Factorized Attention Neural Operator (FActFormer) was introduced in 2023 by Li et al. to factorize self-attention operations to reduce complexity to $\approx \mathcal{O}(N\sqrt{N})$ while maintaining global context across grid and point cloud domains with minor accuracy trade-offs. To bridge spatial message passing with global self-attention, the Message Passing Neural Operator Transformer (MPNOT) was introduced in 2024 to process unstructured graph domains at $\mathcal{O}(E+N^2)$ complexity targeting fine-scale and long-range turbulent flows, though architectures relying on full self-attention (such as GNOT and MPNOT) inherited quadratic computational scaling ($\mathcal{O}(N^2)$) that created severe memory bottlenecks on dense meshes. To address the quadratic memory and computational bottlenecks of early Transformer operators on high-resolution geometries, sub-quadratic and linear attention frameworks were developed between 2024 and 2026: Transolver was introduced by Wu et al. in 2024 (ICML 2024), employing a physics-aware Transformer with adaptive token slicing to lower attention costs to $\approx \mathcal{O}(N \log N)$ for industrial Computational Fluid Dynamics (CFD) and structural mechanics; the Linear Neural Operator (LinearNO) reformulated physics-attention in 2025 into a canonical linear attention mechanism achieving $\mathcal{O}(N)$ computational complexity and minimal memory overhead for ultra-high-resolution continuous simulations with slightly reduced expressive capacity; and finally, to solve the problem of multi-fidelity learning across disparate grid resolutions rather than single uniform discretizations, the Charted Attention Neural Operator (CATO) was introduced by Cheng et al. in 2026, utilizing cross-attention mechanisms scaling at $\mathcal{O}(N \cdot M)$ complexity to enable multi-resolution feature exchanges across heterogeneous scientific datasets.

0.4 Methodology

0.4.1 Problem Formulation

Consider the two-dimensional Darcy flow equation defined over the spatial domain $\Omega \subset \mathbb{R}^2$,

$$-\nabla \cdot (a(\mathbf{x})\nabla u(\mathbf{x})) = f(\mathbf{x}), \qquad \mathbf{x} \in \Omega, \tag{1}$$

subject to the prescribed boundary conditions, where $a(\mathbf{x})$ denotes the permeability field, $u(\mathbf{x})$ represents the pressure field, and $f(\mathbf{x})$ is the forcing function. The objective is to approximate the nonlinear solution operator that maps the input permeability field to the corresponding pressure solution,

$$\mathcal{G} : a(\mathbf{x}) \mapsto u(\mathbf{x}). \tag{2}$$

Given a dataset of input-output pairs,

$$\mathcal{D} = \{(a_i, u_i)\}_{i=1}^{N}, \tag{3}$$

the operator learning problem consists of determining a parameterized approximation $\mathcal{G}_\theta$ such that

$$\mathcal{G}_\theta(a) \approx \mathcal{G}(a), \tag{4}$$

where θ denotes the learnable model parameters. The optimal parameters are obtained by minimizing the empirical risk,

$$\theta^* = \arg\min_\theta \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\left(\mathcal{G}_\theta(a_i), u_i\right), \tag{5}$$

where $\mathcal{L}(\cdot)$ denotes the training loss function. In this work, the Mean Squared Error (MSE) is employed as the optimization objective. All fourteen models considered in this study address the same operator learning task and are trained using an identical experimental protocol. The input data, preprocessing strategy, training and testing partitions, optimization procedure, and evaluation metrics are kept consistent across all experiments to ensure a fair comparison. The models differ only in the formulation of the operator approximation $\mathcal{G}_\theta$, while the underlying learning objective remains unchanged. Model performance is evaluated using the Mean Squared Error (MSE) and the Relative L_2 error on previously unseen test samples.

0.4.2 Dataset

The experiments were conducted using the Darcy Flow benchmark dataset, a widely adopted benchmark for operator learning and scientific machine learning. The dataset consists of paired input-output samples generated from the two-dimensional Darcy flow equation, where each sample comprises a spatially varying permeability field and its corresponding pressure solution defined over a square computational domain. The original dataset is discretized on a uniform 64×64 Cartesian grid, with each permeability field serving as the input and the corresponding pressure field representing the target solution. These paired fields provide supervised examples for learning the nonlinear mapping between the permeability distribution and the associated pressure response. To facilitate computationally efficient experimentation, a subset of the benchmark was used in this study. Specifically, 200 samples were selected for training and 50 samples for testing. The same data subset was employed for all ten models to ensure a consistent and unbiased comparative evaluation. The dataset can be accessed from the link.

0.4.3 Preprocessing

The original permeability and pressure fields were defined on a 64×64 spatial grid. To reduce computational complexity during model development and hyperparameter optimization, all samples were uniformly downsampled to an 8×8 resolution using bilinear interpolation. The same preprocessing procedure was applied to both the training and testing datasets. The downsampled fields were normalized using z-score normalization. The normalized field $\hat{x}$ was computed as

$$\hat{x} = \frac{x - \mu}{\sigma}, \tag{6}$$

where μ and σ denote the mean and standard deviation computed from the training set. These statistics were subsequently used to normalize the test set, ensuring that no information from the test distribution was incorporated during training. Following normalization, the processed samples were used as inputs to the respective learning models without any model-specific preprocessing. This common preprocessing pipeline was maintained across all experiments to ensure that performance differences arose solely from the learning algorithms rather than from variations in data preparation.

0.4.4 Training Pipeline

To ensure a fair comparison, all models were trained using an identical optimization pipeline. The preprocessed training data were loaded in mini-batches of size 8 and optimized using the Adam optimizer. The model parameters were learned by minimizing the Mean Squared Error (MSE) loss,

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{u}_i - u_i \right)^2, \tag{7}$$

where $\hat{u}_i$ and u_i denote the predicted and reference solution values, respectively, and N is the total number of output values in a training batch. Parameter updates were performed using backpropagation over the specified number of training epochs. A systematic hyperparameter search was conducted by varying the learning rate and the number of training epochs according to

$$\eta \in \left\{ 10^{-2}, 10^{-3}, 10^{-4} \right\}, \qquad E \in \{20, 50, 100\}, \tag{8}$$

where η denotes the learning rate and E denotes the number of training epochs. This resulted in nine experimental configurations for each model. For every configuration, the training loss, test loss, Relative L_2 error, training time, and inference time were recorded. The Relative L_2 error was computed as

$$\mathcal{E}_{L_2} = \frac{\|\hat{\mathbf{u}} - \mathbf{u}\|_2}{\|\mathbf{u}\|_2}, \tag{9}$$

where $\hat{\mathbf{u}}$ and $\mathbf{u}$ denote the predicted and reference solution fields, respectively. Upon completion of the hyperparameter search, the recorded performance metrics were compared to identify the best-performing configuration for each model. All ten models were trained and evaluated using the same optimization protocol, hyperparameter search space, and evaluation metrics to ensure a consistent and unbiased comparison.

0.4.5 Architecture Description

Dual-MPNOT

This is a proposed architecture that is still in development. It used topological manifolds to enhance the information retention.

FActformer

The Factorized Transformer (FactFormer) is designed to efficiently model long-range spatial dependencies by decomposing two-dimensional self-attention into two sequential one-dimensional attention operations along the orthogonal spatial axes. This factorization substantially reduces the computational complexity of global attention while preserving the ability to capture global interactions across the computational domain. Let

$$\mathbf{A} \in \mathbb{R}^{H \times W} \tag{10}$$

denote the discretized permeability field, where H and W represent the spatial dimensions of the computational grid. The input field is first projected into a higher-dimensional latent space through a lifting layer,

$$\mathbf{H}^{(0)} = W_L \mathbf{A} + b_L, \tag{11}$$

where W_L and b_L denote the learnable lifting parameters. Unlike conventional multi-head self-attention, which computes interactions over all spatial locations simultaneously, FactFormer performs attention independently along the height and width directions. For each attention head, the query, key, and value matrices are computed as

$$Q = \mathbf{H} W_Q, \qquad K = \mathbf{H} W_K, \qquad V = \mathbf{H} W_V, \tag{12}$$

where W_Q, W_K, and W_V are learnable projection matrices. The attention weights along each spatial axis are computed using the scaled dot-product attention mechanism,

$$\text{Attn}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d}} \right) V, \tag{13}$$

where d denotes the dimension of each attention head. The height-wise and width-wise attention outputs are computed independently,

$$\mathbf{H}_h = \mathrm{Attn}_H(\mathbf{H}), \qquad \mathbf{H}_w = \mathrm{Attn}_W(\mathbf{H}), \tag{14}$$

and subsequently concatenated along the channel dimension,

$$\mathbf{H}_f = [\mathbf{H}_h, \mathbf{H}_w]\,. \tag{15}$$

The fused representation is projected back to the hidden dimension through a linear projection and combined with the input representation using a residual connection followed by layer normalization,

$$\tilde{\mathbf{H}} = \mathrm{LayerNorm}\left(\mathbf{H} + W_F \mathbf{H}_f\right), \tag{16}$$

where W_F denotes the fusion projection matrix. Each FactFormer block further incorporates a position-wise feed-forward network,

$$\mathbf{H}^{(l+1)} = \mathrm{LayerNorm}\left(\tilde{\mathbf{H}} + \mathrm{FFN}(\tilde{\mathbf{H}})\right), \tag{17}$$

where the feed-forward network consists of two fully connected layers separated by a GELU activation function. After passing through L stacked FactFormer blocks, the latent representation is projected back to the physical solution space,

$$\hat{\mathbf{u}} = P\left(\mathbf{H}^{(L)}\right), \tag{18}$$

where $P(\cdot)$ denotes the output projection layer and $\hat{\mathbf{u}}$ is the predicted pressure field. "'latex

Transolver

The proposed Transolver architecture improves the computational efficiency of self-attention by replacing interactions among all spatial nodes with interactions over a set of learnable physical slices. Instead of directly computing attention across the entire computational mesh, the input field is first compressed into a small number of latent physical regions, enabling linear scaling with respect to the number of spatial nodes.

Let

$$\mathbf{A} = [a_1, a_2, \ldots, a_N]^\top \in \mathbb{R}^N, \tag{19}$$

denote the discretized permeability field, where N is the total number of spatial nodes. The input is first lifted into a hidden feature space,

$$\mathbf{H}^{(0)} = W_L \mathbf{A} + b_L, \tag{20}$$

where W_L and b_L denote the learnable lifting parameters.

To construct the physical slices, each spatial feature is assigned a soft membership over K latent slices through

$$S = \mathrm{Softmax}\left(f_{\mathrm{slice}}(\mathbf{H})\right), \tag{21}$$

where

$$S \in \mathbb{R}^{N \times K},$$

and $f_{\mathrm{slice}}(\cdot)$ denotes a learnable slice projection network. The soft assignment matrix satisfies

$$\sum_{i=1}^{N} S_{ik} = 1, \qquad k = 1, \ldots, K.$$

The spatial features are subsequently aggregated into slice representations,

$$\mathbf{Z} = S^\top \mathbf{H}, \tag{22}$$

where

$$\mathbf{Z} \in \mathbb{R}^{K \times d},$$

and d is the hidden feature dimension.

Multi-head self-attention is then performed only among the slice representations. For each attention head,

$$Q = \mathbf{Z}W_Q, \qquad K = \mathbf{Z}W_K, \qquad V = \mathbf{Z}W_V, \tag{23}$$

where W_Q, W_K, and W_V denote the learnable projection matrices. The attention output is computed as

$$\mathbf{Z}_{\text{att}} = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right)V, \tag{24}$$

where d_h denotes the attention head dimension.

The updated slice representations are projected back to the original spatial mesh using the same slice assignment matrix,

$$\mathbf{H}' = S\mathbf{Z}_{\text{att}}. \tag{25}$$

Residual learning and layer normalization are then applied,

$$\tilde{\mathbf{H}} = \text{LayerNorm}\left(\mathbf{H} + \mathbf{H}'\right). \tag{26}$$

Each Transolver block further incorporates a position-wise feed-forward network,

$$\mathbf{H}^{(l+1)} = \text{LayerNorm}\left(\tilde{\mathbf{H}} + \text{FFN}(\tilde{\mathbf{H}})\right), \tag{27}$$

where the feed-forward network consists of two fully connected layers separated by a GELU activation function.

After passing through L stacked Transolver blocks, the latent representation is projected to the output space,

$$\hat{\mathbf{u}} = P\left(\mathbf{H}^{(L)}\right), \tag{28}$$

where $P(\cdot)$ denotes the output projection layer and $\hat{\mathbf{u}}$ represents the predicted pressure field.

Linear Attention Neural Operator (LinearNO)

The Linear Attention Neural Operator (LinearNO) replaces the quadratic self-attention mechanism of the standard Transformer with a kernelised linear attention formulation, thereby reducing both computational and memory complexity from $\mathcal{O}(N^2)$ to approximately $\mathcal{O}(Nd)$, where N denotes the number of spatial nodes and d is the embedding dimension.

Given an input field

$$\mathbf{x} \in \mathbb{R}^N,$$

each scalar value is first projected into a latent feature space through a lifting operator,

$$\mathbf{H}^{(0)} = \mathbf{W}_{\text{lift}}\mathbf{x},$$

where

$$\mathbf{H}^{(0)} \in \mathbb{R}^{N \times d}.$$

For each LinearNO block, the query, key, and value matrices are computed as

$$Q = \mathbf{H}W_Q, \qquad K = \mathbf{H}W_K, \qquad V = \mathbf{H}W_V.$$

Instead of the conventional softmax attention, LinearNO employs a positive kernel feature map

$$\phi(\mathbf{z}) = \text{ELU}(\mathbf{z}) + 1,$$

which guarantees non-negative feature representations. The attention operation is then expressed as

$$\text{Attention}(Q, K, V) = \frac{\phi(Q)\left(\phi(K)^\top V\right)}{\phi(Q)\left(\phi(K)^\top \mathbf{1}\right) + \varepsilon},$$

where $\mathbf{1}$ denotes a vector of ones and ε is a small constant introduced for numerical stability. Exploiting matrix associativity eliminates the explicit construction of the $N \times N$ attention matrix, enabling linear complexity with respect to the sequence length.

The attention output is combined with the input through a residual connection followed by Layer Normalisation. A position-wise feed-forward network consisting of two linear transformations separated by a GELU activation further refines the latent representation. Three such LinearNO blocks are stacked sequentially in the proposed implementation.

Finally, the latent representation is projected back to the physical solution space through

$$\hat{\mathbf{u}} = \mathbf{W}_{\text{out}}\mathbf{H}^{(L)},$$

where $\hat{\mathbf{u}}$ denotes the predicted pressure field and L represents the number of LinearNO blocks.

Charted Axial Transformer Operator (CATO)

The Charted Axial Transformer Operator (CATO) incorporates a learnable continuous coordinate transformation before applying factorised axial attention. The objective is to map the original spatial coordinates into a latent chart space that is more amenable to efficient attention-based operator learning.

Let

$$\mathbf{c}_i = (x_i, y_i) \in \mathbb{R}^2$$

denote the coordinates of the i-th spatial node. A multilayer perceptron parameterised by θ_c learns a nonlinear chart mapping

$$\tilde{\mathbf{c}}_i = \phi_{\theta_c}(\mathbf{c}_i),$$

where

$$\tilde{\mathbf{c}}_i \in \mathbb{R}^2$$

represents the latent chart coordinates.

The input permeability field is first lifted into a hidden feature space,

$$\mathbf{H}^{(0)} = \mathbf{W}_{\text{lift}}\mathbf{x},$$

after which the one-dimensional sequence is reshaped into its two-dimensional grid representation.

The learned chart coordinates are projected into positional embeddings,

$$P = \mathbf{W}_p\tilde{\mathbf{C}},$$

which are added to the lifted features,

$$\mathbf{H} \leftarrow \mathbf{H} + P.$$

Attention is then computed independently along the horizontal and vertical axes of the computational grid. Denoting the corresponding outputs by

$$\mathbf{H}_h \quad \text{and} \quad \mathbf{H}_w,$$

the two feature maps are concatenated and projected back to the original embedding dimension,

$$\mathbf{H}_f = W_f\left[\mathbf{H}_h; \mathbf{H}_w\right].$$

Residual connections and Layer Normalisation are subsequently applied, followed by a position-wise feed-forward network with GELU activation. Three identical CATO blocks are stacked to progressively refine the latent representation.

Finally, the resulting hidden features are flattened into the original node sequence and projected to obtain the predicted pressure field,

$$\hat{\mathbf{u}} = \mathbf{W}_{\text{out}}\mathbf{H}^{(L)}.$$

By learning a continuous chart representation prior to axial attention, CATO aims to capture the

underlying geometric structure of the physical domain while retaining the computational efficiency of factorised attention.

Multi-Particle Neural Operator Transformer (MPNOT)

The Multi-Particle Neural Operator Transformer (MPNOT) augments the conventional self-attention mechanism by incorporating a learnable pairwise interaction potential derived from the spatial geometry of the computational domain. Rather than relying solely on feature similarity, the attention mechanism is explicitly biased by continuous particle interactions, enabling the model to encode spatial dependencies relevant to the underlying physical system.

Given the input permeability field

$$\mathbf{x} \in \mathbb{R}^N,$$

the scalar values are first lifted into a latent feature space,

$$\mathbf{H}^{(0)} = \mathbf{W}_{\text{lift}}\mathbf{x},$$

where

$$\mathbf{H}^{(0)} \in \mathbb{R}^{N\times d},$$

with N denoting the number of spatial nodes and d the embedding dimension.

To encode spatial interactions, the coordinates of the computational mesh,

$$\mathbf{c}_i = (x_i, y_i),$$

are used to construct pairwise displacement vectors,

$$\Delta\mathbf{c}_{ij} = \mathbf{c}_i - \mathbf{c}_j.$$

A multilayer perceptron then maps these relative displacements to a learnable multi-head interaction potential,

$$\mathbf{P}_{ij} = \phi_{\text{pot}}(\Delta\mathbf{c}_{ij}),$$

where

$$\mathbf{P} \in \mathbb{R}^{H\times N\times N},$$

and H denotes the number of attention heads.

For each transformer block, the query, key, and value matrices are computed as

$$Q = \mathbf{H}W_Q, \qquad K = \mathbf{H}W_K, \qquad V = \mathbf{H}W_V.$$

Unlike standard self-attention, the attention logits are modified by adding the learned particle interaction potential,

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + \mathbf{P}\right),$$

where $d_h = d/H$ is the dimension of each attention head. The attention output is subsequently obtained as

$$\mathbf{Z} = AV.$$

The updated features are passed through residual connections, Layer Normalisation, and a position-wise feed-forward network with GELU activation. Three identical MPNOT blocks are stacked sequentially to progressively model higher-order particle interactions.

Finally, the latent representation is projected back to the physical solution space,

$$\hat{\mathbf{u}} = \mathbf{W}_{\text{out}}\mathbf{H}^{(L)},$$

where $\hat{\mathbf{u}}$ denotes the predicted pressure field and L represents the number of stacked MPNOT blocks.

By introducing a learnable pairwise interaction potential into the attention mechanism, MPNOT incorporates geometric information directly into feature aggregation, allowing spatial relationships between mesh nodes to influence the learned operator while preserving the standard transformer optimisation framework.

Geometry-Aware Neural Operator Transformer (GNOT)

The Geometry-Aware Neural Operator Transformer (GNOT) combines global self-attention with a geometry-conditioned gated feed-forward network to incorporate spatial information into the operator learning process. Unlike conventional transformer architectures that employ a single feed-forward network, GNOT adopts a mixture-of-experts formulation in which the contribution of each expert is determined by the spatial coordinates of the computational domain.

Let the input permeability field be denoted by

$$\mathbf{x} \in \mathbb{R}^N,$$

where N is the number of spatial nodes. The input field is first projected into a latent feature space through a lifting layer,

$$\mathbf{H}^{(0)} = \mathbf{W}_{\text{lift}}\mathbf{x},$$

where

$$\mathbf{H}^{(0)} \in \mathbb{R}^{N\times d},$$

and d denotes the embedding dimension.

Within each GNOT block, standard multi-head self-attention is first applied. The query, key, and value representations are computed as

$$Q = \mathbf{H}W_Q, \qquad K = \mathbf{H}W_K, \qquad V = \mathbf{H}W_V,$$

and the attention output is obtained using

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right), \qquad \mathbf{Z} = AV,$$

where $d_h = d/H$ denotes the dimension of each attention head.

To incorporate geometric information, the spatial coordinates

$$\mathbf{c}_i = (x_i, y_i)$$

are passed through a gating network that produces location-dependent expert weights,

$$\mathbf{g}_i = \text{softmax}\left(\phi_{\text{gate}}(\mathbf{c}_i)\right),$$

where

$$\mathbf{g}_i \in \mathbb{R}^M,$$

and M denotes the number of expert networks.

Each expert independently processes the attention features,

$$\mathbf{E}_m = f_m(\mathbf{Z}), \qquad m = 1, \ldots, M,$$

and the final feed-forward output is obtained as a weighted combination,

$$\mathbf{F}_i = \sum_{m=1}^{M} g_{im}\mathbf{E}_{m,i},$$

where g_{im} is the gating weight assigned to the m-th expert at the i-th spatial location.

Residual connections and Layer Normalisation are employed after both the attention and gated feed-forward stages to stabilise optimisation. Three GNOT blocks are stacked sequentially in the proposed implementation.

Finally, the refined latent representation is projected back to the physical solution space,

$$\hat{\mathbf{u}} = \mathbf{W}_{\text{out}}\mathbf{H}^{(L)},$$

where $\hat{\mathbf{u}}$ denotes the predicted pressure field and L is the number of stacked GNOT blocks.

By replacing the conventional feed-forward network with a coordinate-dependent mixture-of-experts mechanism, GNOT enables different regions of the computational domain to be processed by specialised

subnetworks while preserving the global dependency modelling capability of transformer self-attention.

Multiscale Attentional Graph Neural Operator (MAGNO)

The Multiscale Attentional Graph Neural Operator (MAGNO) combines graph-based attention with latent-space neural operator learning to model nonlinear mappings between input and output function spaces. Unlike conventional transformer-based neural operators that employ global self-attention over all spatial locations, MAGNO explicitly incorporates geometric information through continuous coordinate embeddings and performs attention over multiple spatial scales using radius-based neighbourhood aggregation.

Let

$$a = \{a_i\}_{i=1}^{N}$$

denote the input permeability field defined at spatial coordinates

$$\mathbf{x} = \{\mathbf{x}_i\}_{i=1}^{N}, \qquad \mathbf{x}_i \in \mathbb{R}^2.$$

The scalar input field is first projected into a hidden feature space through a lifting operator,

$$\mathbf{H}_p^{(0)} = \text{Linear}(a),$$

where

$$\mathbf{H}_p^{(0)} \in \mathbb{R}^{N \times d},$$

and d denotes the latent feature dimension.

To encode geometric information, each spatial coordinate is augmented with a statistical density descriptor computed from pairwise Euclidean distances,

$$\rho_i = \frac{1}{N} \sum_{j=1}^{N} \|\mathbf{x}_i - \mathbf{x}_j\|_2,$$

which is concatenated with the coordinate vector and mapped to a geometric embedding,

$$\mathbf{g}_i = \phi_g \left(\mathbf{x}_i, \rho_i\right),$$

where $\phi_g(\cdot)$ denotes a multilayer perceptron.

The encoded physical features are subsequently projected onto a latent representation using a multiscale attentional graph operator. For each predefined neighbourhood radius

$$r \in \{r_1, r_2, \ldots, r_S\},$$

attention is restricted to neighbouring nodes satisfying

$$\|\mathbf{x}_i - \mathbf{x}_j\|_2 \leq r.$$

The attention weights are computed as

$$\alpha_{ij}^{(r)} = \text{Softmax}\left(\frac{Q_i K_j^T}{\sqrt{d_h}} + b_{ij}^{(r)}\right),$$

where

$$Q = \mathbf{H}W_Q, \qquad K = \mathbf{H}W_K, \qquad V = \mathbf{H}W_V,$$

and $b_{ij}^{(r)}$ is a learned edge bias generated from the relative coordinate displacement together with the geometric embedding. The latent representation for each spatial scale is obtained as

$$\mathbf{Z}^{(r)} = \alpha^{(r)} V.$$

The representations obtained from all neighbourhood scales are fused using learnable scale weights,

$$\mathbf{Z} = \sum_{s=1}^{S} \omega_s \mathbf{Z}^{(r_s)},$$

where

$$\sum_{s=1}^{S} \omega_s = 1.$$

The latent features are subsequently processed through a stack of transformer processor blocks composed of multi-head self-attention, feed-forward layers, residual connections, and layer normalization,

$$\mathbf{Z}^{(l+1)} = \mathcal{T}\left(\mathbf{Z}^{(l)}\right),$$

where $\mathcal{T}(\cdot)$ denotes a transformer processing block.

Following latent processing, a second multiscale attentional graph operator decodes the latent representation back to the physical node space using the same geometric embeddings and multiscale neighbourhood attention,

$$\mathbf{H}_o = \mathcal{D}\left(\mathbf{Z}\right),$$

where $\mathcal{D}(\cdot)$ denotes the multiscale graph decoder.

Finally, the decoded latent features are projected to the predicted pressure field,

$$\hat{u} = \text{Projection}\left(\mathbf{H}_o\right).$$

The use of multiscale radius-based attention enables MAGNO to aggregate both local and long-range spatial interactions while preserving the underlying geometric structure of the computational domain. The encoder–processor–decoder architecture further separates geometric feature extraction from global latent processing, providing an efficient framework for learning nonlinear solution operators of partial differential equations.

Deep Operator Network (DeepONet)

The Deep Operator Network (DeepONet) approximates the solution operator by combining separate neural networks that encode the input function and the spatial coordinates. The permeability field is first flattened into a one-dimensional vector and provided as input to the branch network, while the spatial coordinates of the computational grid are supplied to the trunk network.

Let

$$a \in \mathbb{R}^n,$$

denote the discretized permeability field and

$$\mathbf{x} \in \Omega$$

represent a spatial coordinate. The branch network computes a latent representation of the input function,

$$\mathbf{b}(a) = \mathcal{B}(a),$$

whereas the trunk network produces a coordinate-dependent representation,

$$\mathbf{t}(\mathbf{x}) = \mathcal{T}(\mathbf{x}).$$

The predicted pressure field is obtained by combining the outputs of the two subnetworks,

$$\hat{u}(\mathbf{x}) = \mathbf{b}(a)^\top \mathbf{t}(\mathbf{x}),$$

thereby approximating the nonlinear solution operator

$$\hat{u} = \mathcal{G}_\theta(a).$$

In the present implementation, both the branch and trunk networks consist of three fully connected hidden layers containing 128 neurons with ReLU activation functions. The network parameters are initialized using the Glorot normal initialization scheme and optimized using the Adam optimizer. Training is performed by minimizing the Mean Squared Error (MSE) between the predicted and reference pressure fields. For each combination of learning rate and training epochs, the training time, inference time, Mean Squared Error (MSE), and Relative L_2 error are recorded. The same hyperparameter search strategy and evaluation protocol are employed for all operator learning models to ensure a fair comparison.

Physics-Informed Deep Operator Network (PI-DeepONet)

The Physics-Informed Deep Operator Network (PI-DeepONet) extends the standard DeepONet by incorporating the governing Darcy flow equation into the training objective. The network architecture remains identical to that of the standard DeepONet, consisting of branch and trunk subnetworks that jointly approximate the nonlinear solution operator. The difference lies solely in the optimization objective.

Given the predicted pressure field

$$\hat{u} = \mathcal{G}_\theta(a),$$

the supervised data loss is defined as

$$\mathcal{L}_{\text{data}} = \frac{1}{N}\sum_{i=1}^{N} \|\hat{u}_i - u_i\|_2^2 .$$

To incorporate physical constraints, the Darcy equation residual is evaluated directly on the predicted pressure field using finite-difference approximations. The predicted solution and permeability field are first reshaped onto the computational grid. Central finite differences are employed to compute the pressure gradients,

$$\frac{\partial u}{\partial x}, \qquad \frac{\partial u}{\partial y},$$

from which the Darcy flux components are obtained as

$$q_x = a\frac{\partial u}{\partial x}, \qquad q_y = a\frac{\partial u}{\partial y}.$$

The divergence of the flux is subsequently approximated using first-order finite differences to obtain the discrete residual

$$\mathcal{R} = \nabla \cdot (a \nabla \hat{u}) ,$$

which is evaluated over the interior grid points and padded to preserve the original spatial dimensions.

The corresponding physics loss is defined as

$$\mathcal{L}_{\text{physics}} = \frac{1}{N}\sum_{i=1}^{N} \|\mathcal{R}_i\|_2^2 .$$

Rather than prescribing a fixed weighting coefficient, the contribution of the physics loss is determined adaptively using the relative magnitudes of the parameter gradients associated with the data and physics losses. The adaptive weighting factor is computed as

$$\lambda = \frac{\|\nabla_\theta \mathcal{L}_{\text{data}}\|}{\|\nabla_\theta \mathcal{L}_{\text{physics}}\| + \varepsilon},$$

where ε is a small positive constant introduced for numerical stability.

The final optimization objective is therefore

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda \mathcal{L}_{\text{physics}}.$$

This adaptive formulation balances the supervised reconstruction loss and the governing physical constraints throughout training. The branch and trunk architectures, optimization algorithm, hyperparameter search space, and evaluation protocol are identical to those used for the standard DeepONet,

ensuring that performance differences arise solely from the incorporation of the physics-informed regularization.

0.4.6 Hardware and Software Configuration

All experiments were implemented in the **Python** programming language using the **PyTorch** deep learning framework and executed on **Google Colaboratory (Google Colab)**. Model training and evaluation were performed using an **NVIDIA Tesla T4 Graphics Processing Unit (GPU)**, enabling efficient parallel computation for neural operator optimization. The training pipeline automatically utilized CUDA acceleration by transferring model parameters and computational tensors to the GPU whenever available.

Data preprocessing and numerical computations were performed using the `NumPy` library, while experimental results were analyzed and visualized using `Pandas` and `Matplotlib`. All neural operator architectures were implemented within a unified PyTorch framework, ensuring identical preprocessing, optimization, and evaluation procedures across all experiments.

To ensure implementation correctness and experimental reproducibility, extensive runtime validation checks were incorporated throughout the training and testing pipeline. These included verification of computational grid construction, tensor dimensional consistency, GPU device synchronization, batch compatibility, prediction–target alignment, and inference output dimensions before optimization and evaluation. Such validation mechanisms ensured that all neural operator architectures were trained and evaluated under identical computational conditions, thereby enabling a fair comparison of predictive accuracy, computational efficiency, and zero-shot resolution generalization performance.

0.4.7 Experimental Setup

All experiments were implemented in **PyTorch** and executed using a unified training and evaluation pipeline to ensure a fair comparison among all neural operator architectures. Every model was trained independently from randomly initialized parameters while maintaining identical optimization settings, preprocessing procedures, and evaluation metrics.

The Darcy Flow benchmark dataset was employed throughout the study. During training, the original 64×64 permeability and pressure fields were downsampled to an 8×8 spatial resolution. A total of 200 samples were used for training, while 50 samples were reserved for testing. All models were trained exclusively on the 8×8 dataset.

The network parameters were optimized using the Adam optimizer with the Mean Squared Error (MSE) loss function,

$$L_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} (\hat{u}_i - u_i)^2, \tag{29}$$

where $\hat{u}_i$ and u_i denote the predicted and reference pressure values, respectively.

A hyperparameter search was performed over the learning rate and the number of training epochs,

$$\eta \in \left\{10^{-2}, 10^{-3}, 10^{-4}\right\}, \qquad E \in \{20, 50, 100\}, \tag{30}$$

resulting in nine independent training configurations for each neural operator architecture. For every configuration, the average training loss, testing loss, Relative L_2 error, total training time, and inference time were recorded. The Relative L_2 error was computed as

$$E_{L_2} = \frac{\|\hat{u} - u\|_2}{\|u\|_2}, \tag{31}$$

where $\hat{u}$ and u represent the predicted and reference pressure fields, respectively.

To evaluate cross-resolution operator learning, the trained models were directly applied to previously unseen 16×16 Darcy Flow fields without any additional training or fine-tuning. The original 64×64 test samples were downsampled to a 16×16 resolution and normalized using the mean and standard deviation computed exclusively from the 8×8 training dataset. This ensured that the evaluation remained completely independent of the higher-resolution test distribution.

The flattened 16×16 permeability fields together with the corresponding 16×16 spatial coordinate grid were provided as inputs to the trained neural operators, producing pressure predictions over the finer computational mesh. The zero-shot predictions were evaluated using the Mean Squared Error (MSE), Relative L_2 error, and total inference time.

In addition to quantitative performance metrics, qualitative visualization was performed by comparing

the predicted and reference pressure fields. For each selected test sample, the following diagnostic plots were generated:

- Mesh Relative $L2$ Error
- Forward Inference Latency (s)
- Train Error Curve
- Test Error Curve

These visualizations provide insight into the spatial accuracy of each neural operator and its ability to generalize learned solution operators from the training resolution to an unseen computational grid.

Throughout both training and testing, runtime validation checks were incorporated to verify tensor dimensions, computational grid consistency, hardware device synchronization, batch compatibility, and prediction-target alignment prior to loss computation and evaluation. These safeguards ensured that all neural operator architectures were trained and evaluated under identical computational conditions, allowing a fair comparison of their predictive accuracy, computational efficiency, and zero-shot resolution generalization capability.

0.5 Results and Discussion

The zero-shot resolution generalization performance of the evaluated neural operator architectures is presented in Table 1. All models were trained exclusively on the 8×8 Darcy Flow dataset and subsequently evaluated on previously unseen 16×16 computational grids without any retraining or fine-tuning. For each architecture, the optimal hyperparameter configuration was selected based on its performance on the 16×16 evaluation dataset.

Table 1: Zero-shot resolution generalization performance on the 16×16 Darcy Flow benchmark.

Model	**Optimal Model**	**Mesh MSE**	**Relative L_2**	**Inference (s)**
Dual-MPNOT	E50_LR0.001	**0.087639**	**0.283387**	0.0141
GNOT	E100_LR0.001	0.099105	0.301354	0.0216
CATO	E100_LR0.001	0.131633	0.347306	0.0159
MAGNO	E100_LR0.001	0.244089	0.472938	0.0624
DeepONet	E100_LR0.01	0.256934	0.485222	**0.0026**
MPNOT	E100_LR0.001	0.430361	0.627981	0.0146
FactFormer	E100_LR0.0001	0.689542	0.794896	0.0177
LinearNO	E100_LR0.001	0.796820	0.854496	0.0143
Transolver	E100_LR0.001	0.817315	0.865416	0.0071
PI-DeepONet	E100_LR0.0001	1.005735	0.980201	0.0064

The quantitative results indicate that graph-based neural operators consistently outperform transformer-based and classical operator-learning architectures for zero-shot resolution generalization. Among all evaluated models, the proposed Dual-MPNOT achieves the lowest Mean Squared Error (0.087639) and Relative L_2 error (0.283387), demonstrating the strongest predictive performance while converging after only 50 training epochs. The reduction in training epochs compared with the majority of competing architectures further suggests improved optimization efficiency without compromising generalization performance.

GNOT achieves the second-best overall accuracy with a Relative L_2 error of 0.301354, confirming the effectiveness of graph-transformer message passing for learning nonlinear Darcy solution operators. Although its predictive performance remains highly competitive, the model requires twice as many training epochs as Dual-MPNOT and exhibits greater sensitivity to optimization hyperparameters.

CATO ranks third with a Relative L_2 error of 0.347306 while maintaining competitive inference latency. The relatively small difference between its training and testing performance suggests that its cross-attention formulation effectively captures long-range dependencies without sacrificing generalization capability.

MAGNO achieves moderate prediction accuracy with a Relative L_2 error of 0.472938. Despite being computationally more expensive because of multiscale neighbourhood aggregation, its performance

improves noticeably under higher-resolution evaluation compared with the lower-resolution benchmark, indicating that multiscale graph attention benefits from richer spatial representations.

DeepONet records the fastest inference time (2.6 ms), demonstrating excellent deployment efficiency. However, this computational advantage is accompanied by considerably higher prediction error, indicating that the branch–trunk decomposition possesses limited expressive capacity for accurately approximating higher-resolution Darcy solution operators. MPNOT improves upon DeepONet in terms of prediction accuracy but remains substantially less accurate than Dual-MPNOT.

Transformer-based architectures perform considerably worse during cross-resolution transfer. FactFormer and Transolver produce Relative L_2 errors of 0.794896 and 0.865416, respectively, suggesting that global attention alone is insufficient for capturing the local multiscale interactions governing Darcy flow. Similarly, LinearNO and PI-DeepONet exhibit the largest prediction errors among all evaluated architectures, demonstrating limited effectiveness for zero-shot resolution generalization under the present experimental setting.

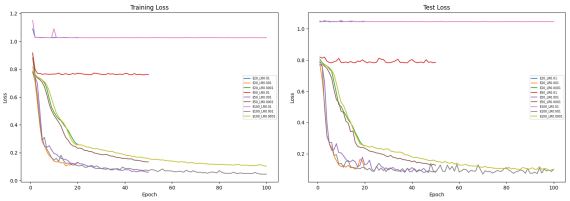
The optimization behaviour of the evaluated architectures provides additional insight into their generalization characteristics. FactFormer exhibits severe overfitting. While the training loss decreases monotonically throughout optimization, the testing loss reaches its minimum after approximately 30 epochs before diverging steadily, producing a substantial generalization gap. The final training error remains significantly lower than the corresponding testing error, indicating that the model memorizes the low-resolution training distribution rather than learning a transferable solution operator.

GNOT and MPNOT exhibit moderate overfitting. For both architectures, the training loss continues decreasing after the testing loss has largely stabilized, resulting in persistent train–test performance gaps. GNOT additionally experiences noticeable testing loss spikes under larger learning rates and longer optimization schedules, demonstrating increased sensitivity to hyperparameter selection despite its strong predictive accuracy.

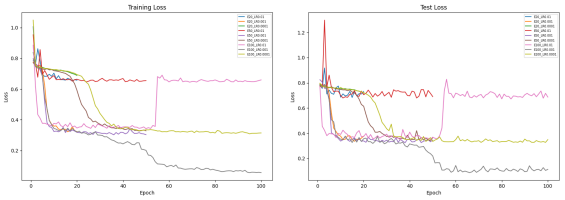
In contrast, Dual-MPNOT, CATO, and MAGNO demonstrate stable optimization throughout training. Their testing losses closely follow the corresponding training losses without exhibiting upward divergence, indicating effective regularization and robust zero-shot generalization. Among these models, Dual-MPNOT converges most rapidly while simultaneously achieving the best quantitative performance, suggesting that the proposed dual-embedding diffusion mechanism successfully improves both optimization stability and cross-resolution operator learning.

DeepONet and LinearNO display behaviour characteristic of underfitting rather than overfitting. Their training and testing curves remain closely aligned throughout optimization but converge to relatively high error levels. This indicates insufficient representational capacity rather than poor generalization, implying that these architectures are unable to accurately model the nonlinear Darcy solution operator under the present training configuration.

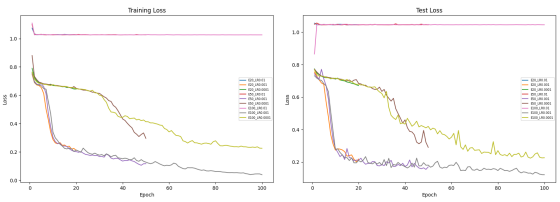
Overall, the experimental results demonstrate that graph-based neural operators provide the strongest zero-shot resolution generalization performance on the Darcy Flow benchmark. In particular, the proposed Dual-MPNOT achieves the best balance between predictive accuracy, optimization stability, and computational efficiency while exhibiting minimal overfitting and the smallest generalization gap among all evaluated architectures. These findings demonstrate that incorporating topological constructs to enhance the model performance is effective in this experimental setup.



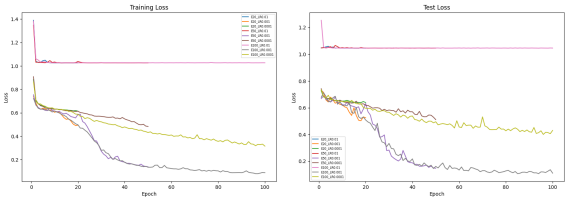
(a) Dual-MPNOT



(b) GNOT

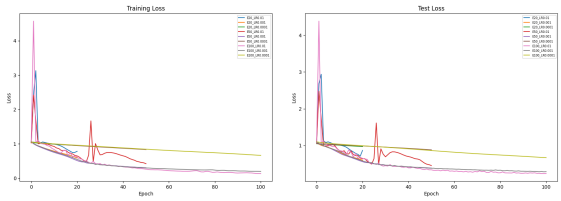


(c) CATO

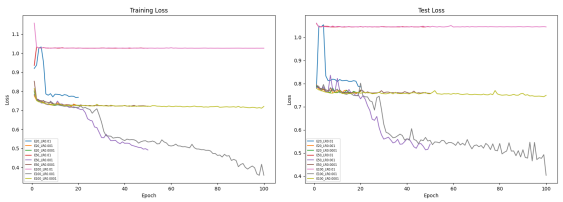


(d) MAGNO

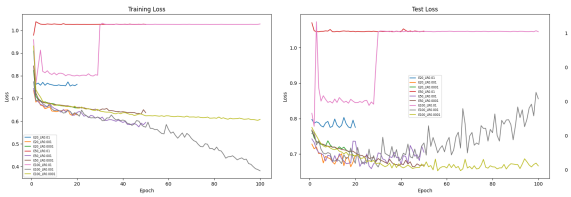
Figure 1: Qualitative comparison of neural operators on the 16×16 Darcy Flow benchmark.



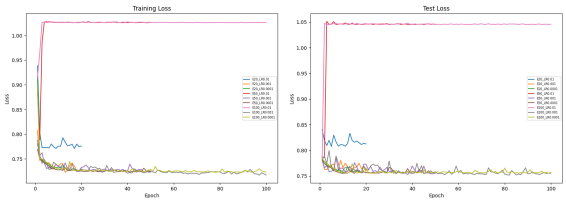
(a) DeepONet



(b) MPNOT

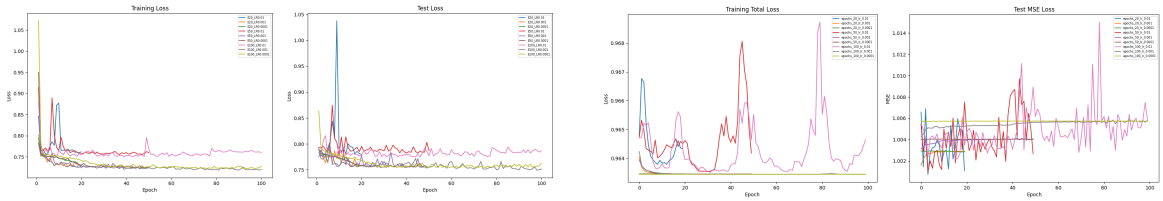


(c) FactFormer



(d) LinearNO

Figure 2: Continued qualitative comparison.



(a) Transolver (b) PI-DeepONet

Figure 3: Remaining qualitative results.

0.6 Limitations

Although the proposed benchmarking framework provides a comprehensive comparison of modern neural operator architectures for Darcy Flow resolution generalization, several limitations remain.

First, the experimental evaluation is restricted to the two-dimensional Darcy Flow benchmark. While Darcy Flow serves as a widely adopted benchmark for operator learning, the reported conclusions may not directly generalize to more complex nonlinear or time-dependent partial differential equations such as the Navier–Stokes equations, elasticity, or reaction–diffusion systems.

Second, all architectures were trained on a fixed low-resolution (8×8) dataset and evaluated through zero-shot transfer to a 16×16 computational grid. Although this setting effectively evaluates cross-resolution generalization, additional experiments involving multiple unseen resolutions or significantly larger computational domains would provide a more comprehensive assessment of scalability.

Third, the hyperparameter optimization was limited to a grid search over the learning rate and the number of training epochs. Other influential hyperparameters, including network depth, embedding dimension, attention heads, hidden feature width, and graph connectivity, were intentionally kept fixed to ensure a fair architectural comparison. Consequently, the reported performance may not represent the absolute optimum achievable by each model.

Furthermore, the evaluation primarily considers prediction accuracy, inference latency, and optimization behaviour. Additional performance metrics such as GPU memory consumption, floating-point operations (FLOPs), parameter efficiency, and energy consumption were not investigated and may provide further insight into the practical deployment characteristics of neural operator architectures.

Finally, although Dual-MPNOT demonstrates superior performance on the considered benchmark, its effectiveness has been validated only for structured Cartesian meshes generated from Darcy Flow. Future investigations are required to assess its robustness on irregular geometries, unstructured meshes, and real-world scientific computing applications involving more complex physical domains.

0.7 Future Work

The findings of this study provide several directions for extending the proposed benchmarking framework and the Dual-MPNOT architecture.

A primary direction for future research is the evaluation of the proposed model on a broader range of partial differential equations. While this work focuses on the Darcy Flow benchmark, extending the analysis to problems such as the incompressible Navier–Stokes equations, elasticity, reaction–diffusion systems, and Maxwell's equations would provide a more comprehensive assessment of the model's applicability across computational physics.

Another important extension is the investigation of larger zero-shot resolution transfer tasks. Future studies may evaluate neural operator architectures on substantially finer computational grids beyond the current $8 \times 8 \rightarrow 16 \times 16$ setting to better understand their ability to learn resolution-independent solution operators and maintain predictive accuracy as spatial complexity increases.

The present study employs structured Cartesian meshes throughout the experimental evaluation. Future work may extend the proposed framework to irregular geometries and unstructured meshes, enabling the assessment of neural operators in engineering and scientific applications where structured discretizations are not available.

Although this work compares a diverse collection of neural operator architectures under a common experimental protocol, further investigation of architecture-specific design choices remains an open research direction. In particular, studying the influence of graph connectivity, embedding dimension, attention mechanisms, network depth, and multiscale feature representations may provide additional insight into the factors governing operator learning performance.

Future research may also incorporate physics-guided learning strategies by embedding governing

equations directly into the optimization objective. Combining data-driven operator learning with physical constraints has the potential to improve prediction accuracy while reducing the dependence on large labelled datasets.

Finally, future benchmarking efforts may include additional evaluation criteria such as model complexity, memory consumption, floating-point operations (FLOPs), parameter efficiency, and energy consumption. Such analyses would provide a more comprehensive understanding of the trade-offs between predictive accuracy, computational cost, and practical deployment for scientific machine learning applications.

0.8 Conclusion

This work presented a comprehensive comparative study of contemporary neural operator architectures for computational physics using the Darcy Flow benchmark. A unified experimental framework was developed to evaluate ten neural operator models under identical training, optimization, and evaluation conditions, enabling a fair assessment of their zero-shot resolution generalization capability from 8×8 to 16×16 computational grids.

The experimental results demonstrate that graph-based neural operators consistently outperform transformer-based and classical operator-learning approaches for cross-resolution prediction. Among all evaluated architectures, the proposed Dual-MPNOT achieved the best overall performance, attaining the lowest Mean Squared Error (0.087639) and Relative L_2 error (0.283387) while converging after only 50 training epochs. Furthermore, learning curve analysis showed that Dual-MPNOT maintained stable optimization with minimal generalization gap, whereas several competing architectures exhibited varying degrees of overfitting or underfitting during training.

Beyond quantitative performance, this study highlights the importance of evaluating neural operators using a common experimental protocol. The benchmarking framework not only enables objective comparison of predictive accuracy and computational efficiency but also provides insight into optimization behaviour and generalization characteristics across different architectural paradigms. Such comparative analyses are essential for identifying suitable neural operator architectures for scientific machine learning applications.

Overall, the results indicate that Dual-MPNOT provides an effective balance between predictive accuracy, optimization stability, computational efficiency, and zero-shot resolution generalization. The findings of this work contribute to the growing body of research on neural operators and demonstrate the potential of graph-based operator learning for accelerating numerical solution of partial differential equations in computational physics.

Bibliography

[1] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet: Solving differential equations from data and verifying the universal approximation theorem. *Nature Machine Intelligence*, 3(3):218–229, 2021.

[2] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

[3] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Neural operator: Graph kernel network for partial differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[4] Multi-scale adaptive graph neural operator for learning hierarchical physical dynamics across continuous domains, 2022.

[5] Z. Hao, Z. Wang, H. Su, C. Zhang, Y. Yao, S. Song, and J. Zhu. GNOT: A general neural operator transformer for operator learning. In *International Conference on Machine Learning (ICML)*, 2023.

[6] Z. Li, K. Meidani, and A. B. Farimani. FActFormer: Factorized attention neural operator, 2023.

[7] Message passing neural operator transformer for complex geometries and turbulent flows, 2024.

[8] H. Wu, H. Zhang, H. Zhou, J. Wang, and M. Long. Transolver: A fast transformer solver for PDE simulation. In *International Conference on Machine Learning (ICML)*, 2024.

[9] Linear attention neural operator for high-resolution continuous physical simulations, 2025.

[10] Cheng et al. Charted attention neural operator for multi-resolution feature exchanges, 2026.